

Appendix to “CANcept: Model-based CAN Traffic Generation and Manipulation”

Lino Wertz

Karlsruhe Institute of Technology
Germany

Junes Sheikhi

Karlsruhe Institute of Technology
Germany

Florian Fehrle

Karlsruhe Institute of Technology
Germany

Adrian Rupp

Karlsruhe Institute of Technology
Germany

Tianhai Liu*

Karlsruhe Institute of Technology
Germany

Philipp Kern

Karlsruhe Institute of Technology
Germany

Carsten Sinz

Karlsruhe University of Applied Sciences
Germany

Bernhard Beckert

Karlsruhe Institute of Technology
Germany

1 Purpose

This document is the public appendix for the MODELS 2026 Tools and Demonstrations camera-ready submission of CANcept. It contains the supplementary artifact, reproducibility, demonstration, and screencast material that is not part of the 5-page published paper.

2 Artifact and Reproducibility

CANcept is archived on Zenodo <https://doi.org/10.5281/zenodo.21385303> and its repository is on GitHub <https://github.com/CANcept/CANcept>. The artifact includes the source code, benchmark inputs, example DBC files, execution scripts, and reproduction instructions. All reported experiments can be reproduced without physical CAN hardware by using the supplied vcan0 setup. The project is released under the Apache 2.0 license.

The reference environment is a Linux system with SocketCAN support. Building from source requires CMake 3.23 or newer, a C++20 compiler, Qt 6.7.3 or newer, and Ninja as the recommended build backend.

3 Demonstration Scenario

The demonstration scenario for CANcept uses a Linux virtual CAN interface, vcan0, and an example DBC file from the artifact package. It follows a single end-to-end workflow covering CAN interface setup, installation and startup, CAN interface selection, DBC file loading, TSM configuration via the GUI, traffic generation, traffic monitoring, trace recording, replay, and traffic manipulation. Although the scenario uses a virtual CAN interface for reproducibility, the same steps can be executed on a physical SocketCAN-compatible CAN interface.

3.1 Setup CAN Interface

To prepare the environment, create the virtual CAN interface vcan0. These commands must be executed again after each system reboot.

```
$ sudo modprobe vcan
$ sudo ip link add dev vcan0 type vcan
$ sudo ip link set up vcan0
```

*Corresponding author.

If the setup succeeds, the vcan0 interface can be inspected using ifconfig:

```
$ ifconfig vcan0
vcan0: flags=193<UP,RUNNING,NOARP> mtu 72
        unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 1000
        (UNSPEC)
        RX packets 0 bytes 0 (0.0 B)
        RX errors 0 dropped 0 overruns 0 frame 0
        TX packets 0 bytes 0 (0.0 B)
        TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

3.2 Installation and Startup

On Debian-based systems, CANcept can be installed from the project APT repository:

```
$ curl -fsSL
https://raw.githubusercontent.com/CANcept/CANcept/apt-repo/setup.sh | sudo bash
$ sudo apt install CANcept
```

On Arch Linux, CANcept can be installed from the AUR:

```
$ yay -S cancept
```

After installation, start CANcept using the following command:

```
$ CANcept
```

The graphical user interface then opens, as shown in Figure 1.

3.3 Building and Starting from Source

CANcept can also be built from source. On Debian-based systems, install the required dependencies as follows:

```
$ sudo apt update && sudo apt install -y \
qt6-base-dev \
qt6-tools-dev \
qt6-tools-dev-tools \
libqt6svg6-dev \
cmake \
ninja-build \
build-essential \
clang-tidy \
clang-format \
doxygen \
graphviz \
lcov
```

After installing the dependencies, clone the repository and run start.sh, which builds and starts CANcept:

```
$ git clone --recursive https://github.com/CANcept/CANcept.git
$ cd CANcept
$ ./start.sh
```

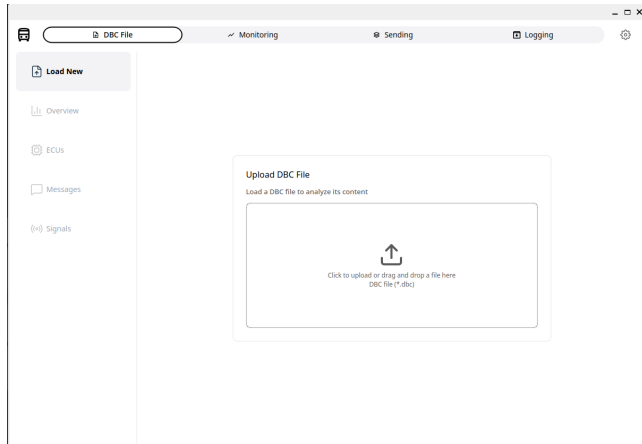


Figure 1: CANcept first window.

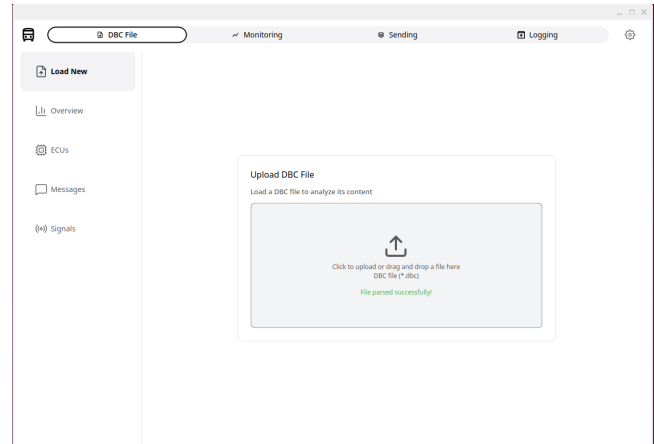


Figure 3: Load a DBC file.

The graphical user interface should then open, as shown in Figure 1.

The GitHub repository <https://github.com/CANcept/CANcept> also provides development and quick-build variants of `start.sh`, together with unit, integration, system, and performance tests. Detailed information about these variants is available in the project README <https://github.com/CANcept/CANcept/blob/main/README.md>.

3.4 CAN Interface Selection

After the GUI opens, the user selects `vcan0` as the active CAN interface, as shown in Figure 2. This binds subsequent sending, monitoring, logging, replay, and manipulation actions to the same SocketCAN endpoint.

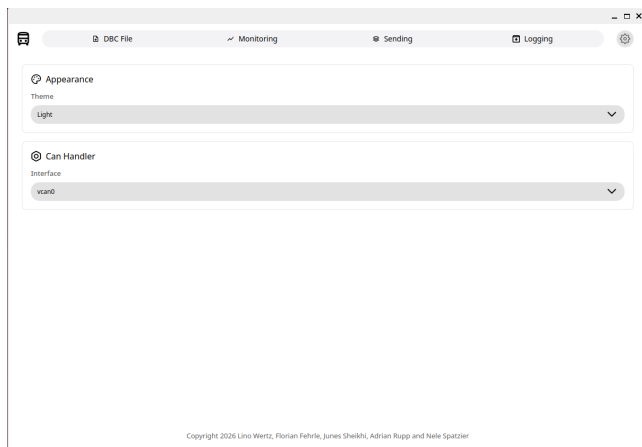


Figure 2: Select the active CAN interface.

3.5 DBC File Loading

With the interface selected, the user loads an example DBC file, as shown in Figure 3. This establishes the DCM used throughout the remainder of the workflow.

After the establishment of DCM, CANcept provides several inspection views that expose the communication model's structure. The overview page in Figure 4 summarizes the loaded DCM artifact.

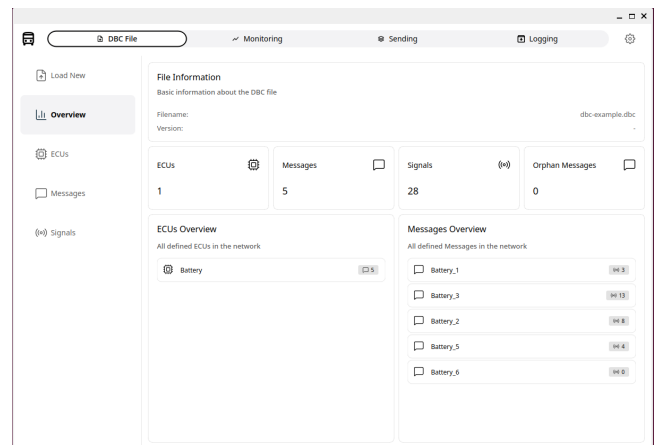


Figure 4: Overview of the DBC-derived DCM.

The ECU view, shown in Figure 5, lists the transmitting and receiving nodes defined in the DCM. This helps the user relate message definitions to the logical components of the CAN network.

The message view in Figure 6 displays the available CAN messages together with identifiers, lengths, and related metadata. This is the main entry point for understanding which frames can later be generated or decoded by the tool.

Finally, the signal view in Figure 7 exposes the signals contained in the selected messages, including value ranges, scaling information, and units. These signal definitions are reused later when configuring TSM-based traffic generation and when decoding monitored or replayed traffic.

3.6 TSM Configuration via the GUI

Next, the user configures a TSM in the GUI, as shown in Figure 8. The TSM specifies which DBC-defined message is sent, at which

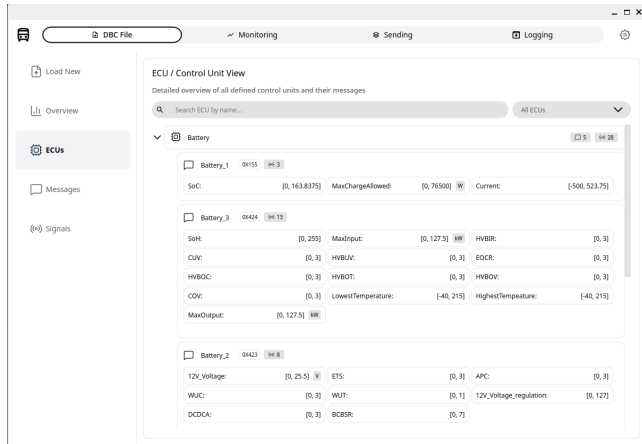


Figure 5: Inspect ECUs defined in the DCM.

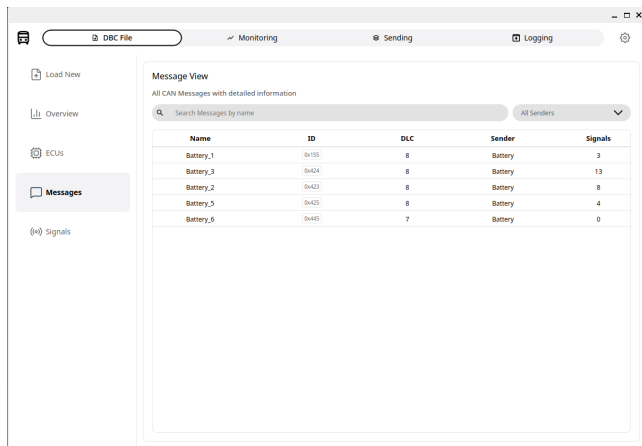


Figure 6: Inspect messages defined in the DCM.

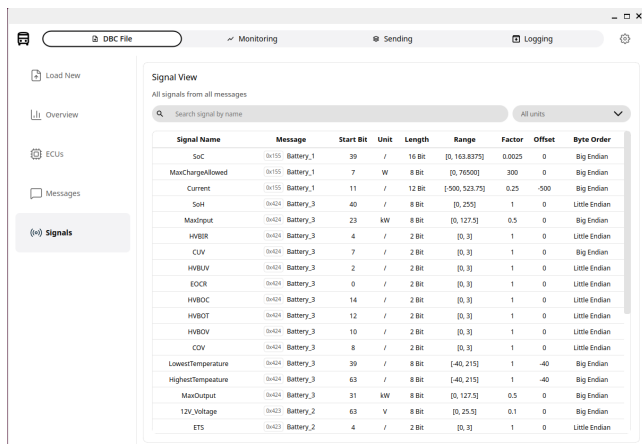


Figure 7: Inspect signals defined in the DCM.

rate, and with which signal values. CANcept resolves the referenced messages and signals against the loaded DCM before execution.

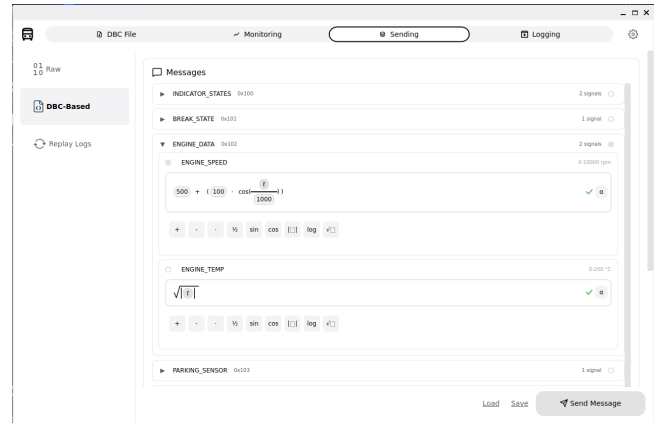


Figure 8: Configure and generate traffic from a TSM.

3.7 Traffic Generation

Once the TSM is configured, the user starts traffic generation from the Sending view, again shown in Figure 8. CANcept encodes the configured message-level values and sends the resulting CAN frames on `vcan0`. This demonstrates that model-level signal assignments are translated into concrete CAN traffic without manual payload construction.

3.8 Traffic Monitoring

The monitoring view, shown in Figure 9, displays incoming CAN frames on `vcan0`. When a matching DCM is loaded, the frames are decoded and shown together with their message- and signal-level interpretations.

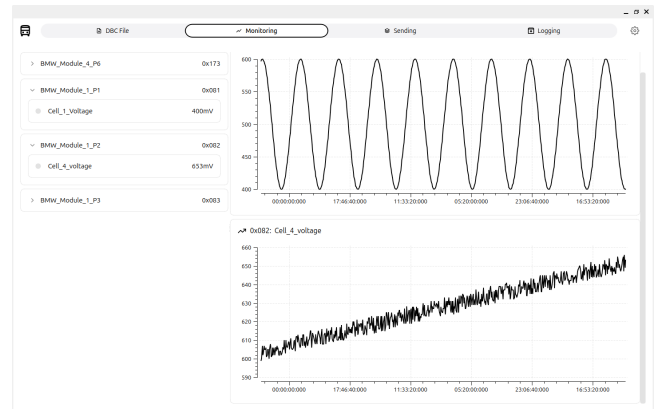


Figure 9: Monitor decoded CAN traffic.

3.9 Trace Recording

Start a logging session while traffic is transmitted on `vcan0`, as shown in Figure 10 and Figure 11. CANcept records the observed traffic as a structured trace. The trace preserves message occurrences, timestamps, raw frame data, and decoded signal values obtained through the DCM. This produces a reusable artifact for inspection, replay, and fault-injection experiments.

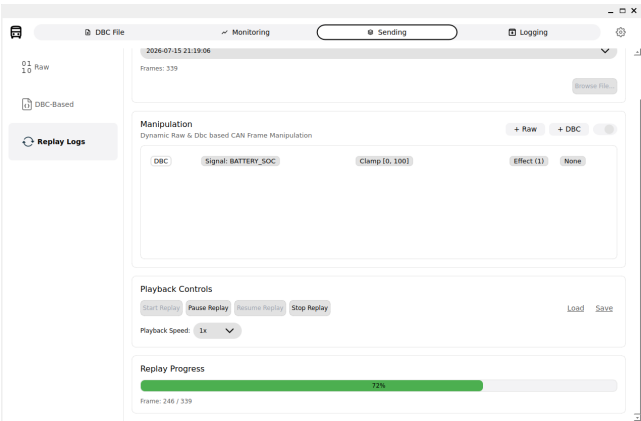


Figure 12: Replay a recorded trace.

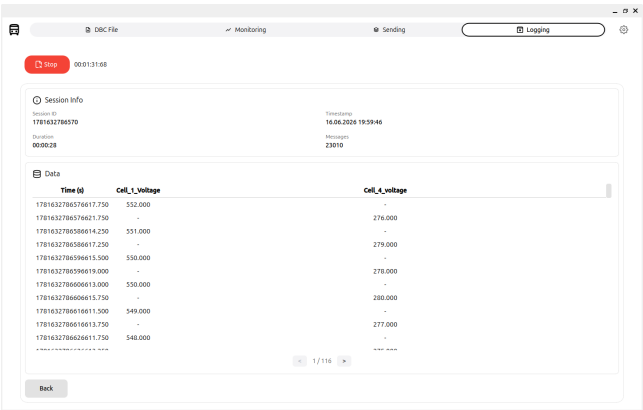


Figure 11: Inspect recorded signal values.

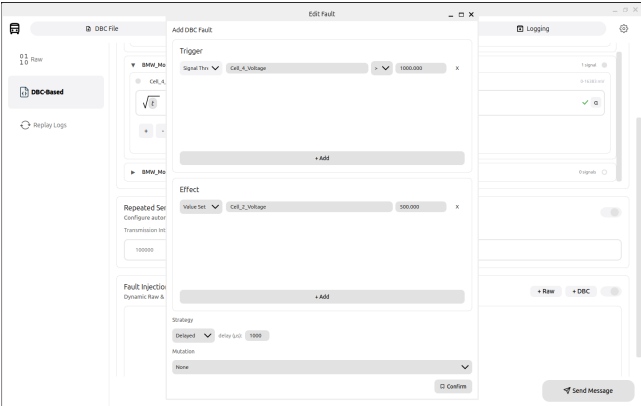


Figure 13: Configure traffic-manipulation rules.

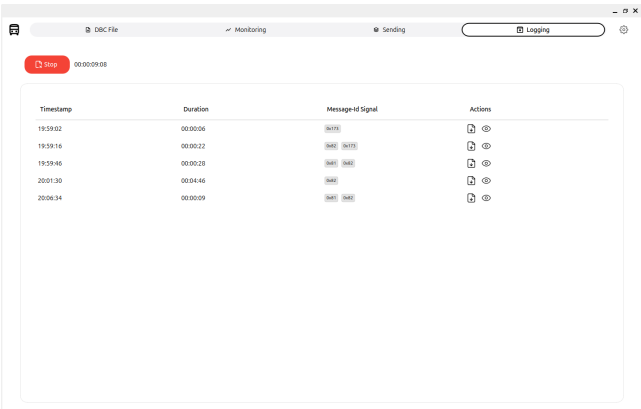


Figure 10: Record CAN traffic as a trace.

3.10 Replay

Select the recorded trace and start replay, as shown in Figure 12. CANcept retransmits the recorded CAN frames on vcan0 according to the timestamps stored in the trace. During replay, the monitoring view decodes the transmitted frames using the same DCM, allowing the user to inspect whether the replayed traffic follows the recorded signal behavior.

3.11 Traffic Manipulation

Finally, configure manipulation rules for the replayed trace, as shown in Figure 13. The rules select message occurrences by conditions over identifiers, signal values, occurrence counts, or timestamps. CANcept applies the configured manipulations to the decoded message-level trace and sends the resulting traffic on vcan0. During the demonstration, the monitoring view is used again to compare the replayed baseline traffic with the manipulated execution and to verify that the configured effects are visible on the bus.

4 Screencast

A screencast of the demonstrated workflow is publicly available on YouTube <https://youtu.be/FpYuuawf2HA> and permanently archived on Zenodo <https://doi.org/10.5281/zenodo.21385303>.